Toward a Formal Framework for Machine Construction: Compositional Architectures, Meta-Machines, and Probabilistic Synthesis

Aslan Alwi, Munirah
Almudaya Research Institute / Almudaya Quantum Labs
Prodi Teknik Informatika, Fakultas Teknik, Universitas Muhammadiyah Ponorogo

***Abstract*—The increasing use of autonomous agents, compositional software systems, and dynamically orchestrated computational components raises a fundamental question: can machines be systematically constructed from other machines through a framework that supports compositionality, formal verification, recursive construction, and probabilistic exploration? Existing computational models, software synthesis techniques, and formal verification methods provide important foundations for addressing parts of this problem, but a unified construction-oriented abstraction remains an open research direction.**

This paper introduces an initial conceptual framework for machine construction in which computational machines are treated not only as entities that execute computations, but also as constructible objects that can be composed into graphs, transformed into new architectures, and used as components of higher-order machine-building systems. Nondeterministic Universal Turing Machines (NUTMs) serve as an inspiration for exploring branching possibilities, without being assumed to constitute the complete mathematical foundation of the proposed framework. The framework distinguishes deductive machine construction from probabilistic synthesis and separates construction mechanisms from external formal verification.

A preliminary architecture is proposed in which a construction agent operates through an explicit construction scaffold, generates machine or graph descriptions, and submits them to independent verification mechanisms before execution. YAML is considered a possible declarative representation for the prototype, rather than the underlying mathematical foundation. The paper also outlines a Proof-of-Concept system, tentatively named Almudaya Machine Construction Agent (AMCA), and defines research questions concerning machine representation, compositional operators, recursive meta-construction, dynamic architecture transformation, and verification of nonlinear computational graphs.

This work is presented as a conceptual framework and research agenda. It does not claim a completed mathematical theory or an experimentally validated construction system. Its purpose is to establish a consistent foundation for subsequent formalization, implementation, and empirical evaluation.

***Index Terms*—machine construction, meta-machines, nondeterministic computation, compositional architecture, probabilistic synthesis, formal verification, computational graphs, agentic systems.**

I. 1. Introduction

Computational systems are increasingly assembled from heterogeneous components, including programs, APIs, autonomous agents, databases, communication protocols, and external services. These components may operate sequentially, concurrently, conditionally, or through dynamically changing dependency structures. In many contemporary systems, the architecture itself is no longer entirely fixed at design time. Components may be generated, replaced, reconfigured, or orchestrated in response to a task or runtime condition.

This development motivates a question that extends beyond the execution of a single program:

Can a computational machine be systematically constructed from other machines, and can the resulting machine itself become a constructor of further machines?

The question becomes more demanding when the target architecture is not restricted to a linear pipeline. A constructed system may contain branches, parallel operations, synchronization points, conditional transitions, shared memory, and mechanisms for modifying its own computational structure. Moreover, the construction process may be deductive, probabilistic, or a combination of both.

The theory of computation provides foundational abstractions for understanding computational machines. A universal Turing machine can simulate another Turing machine given an appropriate encoding of its description [1]. A nondeterministic Turing machine permits multiple possible transitions from a configuration and accepts an input when at least one computation branch reaches an accepting configuration [2]. These concepts offer useful perspectives on universality and branching possibilities.

However, simulation, execution, and construction are distinct concepts. A machine capable of simulating another machine does not, by that fact alone, provide a general framework for synthesizing a new architecture from a specification, proving its compositional properties, or managing the recursive construction of meta-machines.

This paper proposes a construction-oriented perspective in which machines are treated as formal objects that can be represented, composed, transformed, verified, and executed. Nondeterministic Universal Turing Machines (NUTMs) provide an initial source of intuition for exploring alternative possibilities, but the proposed framework is not restricted to Turing-machine representations or to nondeterministic computation.

The paper has five objectives:

1) To establish a conceptual abstraction for constructible computational machines.
2) To distinguish individual machines, machine graphs, and meta-machines.

3) To propose separate but interoperable mechanisms for deductive construction and probabilistic synthesis.
4) To position external formal verification as an independent subsystem.
5) To design a Proof-of-Concept agent that constructs machines and meta-machines through an explicit scaffold.

The intended contribution is an initial conceptual framework and a research agenda. The mathematical definitions and operational mechanisms proposed here require further refinement and evaluation.

A. 1.1. Research Questions

The proposed framework is organized around six questions.

RQ1. Machine representation: How can computational machines be represented as constructible objects independently of their eventual execution environment?

RQ2. Compositional construction: How can multiple machines be composed into nonlinear computational graphs while preserving interface compatibility and specified behavioral properties?

RQ3. Meta-construction: How can a machine construct another machine that may itself possess machine-construction capabilities?

RQ4. Formal verification: How can independently defined verification mechanisms establish the validity of constructed machines, graphs, and architecture transformations?

RQ5. Probabilistic synthesis: How can probabilistic exploration generate alternative machine architectures while retaining explicit validity requirements?

RQ6. Operational realization: How can these abstractions be instantiated in an agentic system using a construction scaffold and a runtime capable of executing constructed architectures?

These questions are related but not equivalent. A successful answer to one does not automatically solve the others.

II. 2. Conceptual Background

A. 2.1. Universal and Nondeterministic Computation

A universal Turing machine is conventionally understood as a machine that simulates other Turing machines through suitable encodings [1]. This provides a general model of computation in which machine descriptions can be treated as data supplied to another machine.

A nondeterministic Turing machine extends the transition model by permitting multiple possible next configurations. Its acceptance condition is existential over computation branches: an input is accepted if at least one branch reaches an accepting configuration [2].

For this research, nondeterminism is used as a conceptual inspiration rather than as a direct implementation prescription.

The relevant intuition is that a computational process can be understood as exploring a space of possible continuations. The proposed framework extends this intuition to a different object: instead of exploring only possible execution paths, a construction system may explore possible machine descriptions and computational architectures.

This distinction is fundamental.

Let a computation configuration be denoted by c. A nondeterministic transition relation can be represented abstractly as:

$$c \to \{c_1, c_2, \ldots, c_n\}.$$

By contrast, a machine-construction process may produce alternative architecture candidates:

$$\mathcal{C}(D) = \{A_1, A_2, \ldots, A_n\},$$

where D is a construction specification and A_i is a candidate machine or architecture.

The first expression concerns possible computational transitions. The second concerns possible constructed objects.

The two may be conceptually related, but the second does not follow automatically from the first. Machine construction requires an additional representation of architectures, construction rules, and the relationship between specifications and candidate machines.

B. 2.2. Program Synthesis and Machine Construction

Program synthesis studies techniques for generating programs from specifications or other forms of intent. Existing surveys describe a range of synthesis methods and application domains [3].

Machine construction, as proposed here, is broader in its target abstraction. The object being constructed need not be a single program. It may be:

- an executable computational component;
- a composition of heterogeneous components;
- a directed graph of interacting machines;
- a dynamically reconfigurable architecture;
- a machine whose function includes constructing other machines.

This distinction is intended as a research direction, not as a claim that existing synthesis research cannot represent such objects. The relationship between the proposed framework and existing work in program synthesis, automated planning, workflow synthesis, and self-adaptive systems must be examined systematically.

The central concern is the construction process as an explicit computational object: its representation, operators, composition rules, verification obligations, and ability to operate recursively.

C. 2.3. Formal Verification as an Independent Discipline

Formal verification provides methods for reasoning about whether a program or system satisfies specified properties. Hoare logic, for example, uses assertions about preconditions and postconditions to reason about program behavior [4].

A Hoare triple has the form:

$$\{P\}\ C\ \{Q\},$$

where P is a precondition, C is a command, and Q is a postcondition.

Such a triple does not automatically express every property of a concurrent or dynamically reconfigurable graph. Nevertheless, Hoare-style reasoning offers a foundation for specifying and verifying individual components and certain forms of composition.

Other verification approaches may be appropriate for different properties, including type checking, model checking, temporal logic, concurrent reasoning, and probabilistic verification.

The proposed framework therefore does not assume that one verification formalism must handle every possible machine architecture. Instead, it treats verification as an external subsystem whose methods are selected according to the properties that must be established.

III. 3. A Construction-Oriented Abstraction of Machines

A. 3.1. Machines as Constructible Objects

Let $\mathfrak{M}$ denote a space of machine descriptions admitted by a particular formal model.

At this stage, $\mathfrak{M}$ is an abstract placeholder. A concrete theory must define the permitted machine representations, their operational semantics, and the conditions under which two descriptions denote equivalent or compatible machines.

A machine description may include:

$$M = (D_M, I_M, O_M, S_M, T_M),$$

where:

- D_M is the machine description;
- I_M specifies input interfaces;
- O_M specifies output interfaces;
- S_M specifies state and memory requirements;
- T_M specifies transition or execution semantics.

This tuple is a provisional modeling proposal rather than a canonical definition of all computational machines. Some machine classes may require additional elements, such as security policies, communication protocols, resource constraints, or probabilistic transition rules.

The important conceptual change is that a machine is represented not only by its executable implementation but also by a description that can be manipulated by a construction system.

B. 3.2. Machine Construction Operators

Let $\mathfrak{D}$ denote a space of construction specifications. A construction operator can initially be represented as:

$$\mathcal{C} : \mathfrak{M} \times \mathfrak{D} \to \mathcal{P}(\mathfrak{M}),$$

where $\mathcal{P}(\mathfrak{M})$ is the power set of $\mathfrak{M}$.

The operator takes a constructor M and a specification D, then produces a set of candidate machine descriptions.

The set-valued representation is intended to accommodate the possibility of multiple candidates. A deterministic constructor may produce a singleton set, whereas a nondeterministic or exploratory constructor may produce multiple candidates.

This notation does not yet establish that every candidate can be generated effectively, that the candidate space is finite, or that construction always terminates. Those properties depend on the formal definition of the constructor and the admitted machine class.

At least four operator families appear necessary for further study:

1) **Generation operators**, which produce candidate machine descriptions.
2) **Composition operators**, which combine machines into larger architectures.
3) **Transformation operators**, which modify an existing machine or architecture.
4) **Meta-construction operators**, which produce constructors or systems capable of further construction.

The algebraic properties of these operators—including associativity, identity, closure, compatibility, and termination—remain open research questions.

C. 3.3. Composition into Computational Graphs

A computational architecture need not be a sequence of operations. It may be represented by a graph:

$$G = (V, E),$$

where V is a set of machine components and E is a set of relations between components.

An edge may represent a data dependency, communication channel, control dependency, or synchronization relationship. These meanings must be distinguished in a concrete semantics.

For example, a graph may represent an input validator that activates two analysis agents in parallel, followed by an aggregation component.

$$I \to V \to \{A_1, A_2\} \to F \to O.$$

Here, I represents input acquisition, V represents validation, A_1 and A_2 represent parallel analyses, F represents aggregation, and O represents output generation.

The graph is more than a visual description. It must carry enough information to determine:

- which components are present;
- how their interfaces are connected;
- which dependencies must be satisfied;
- which execution relationships apply;
- which state or memory resources are shared;
- which constraints govern valid composition.

Some systems may require hypergraphs or other relational structures when a single dependency involves multiple sources or targets. Graphs and hypergraphs are therefore candidate representations, not competing universal solutions.

D. 3.4. Interfaces, Protocols, and Memory

A general construction framework must account for more than computational nodes.

Each component may expose input and output interfaces through mechanisms such as a graphical interface, command-line interface, API, protocol endpoint, sensor, or physical actuator.

Communication between components is governed by protocols and interaction semantics. Each connection may have its own data format, transmission rules, error behavior, and synchronization requirements.

Memory must also be represented explicitly. Relevant forms include internal state, persistent storage, databases, knowledge graphs, retrieved information, and shared working memory.

These elements should not be conflated. For example, retrieval-augmented generation is a retrieval architecture, not itself a storage medium.

A concrete machine description may therefore include an interface contract, a communication contract, a memory model, and resource constraints. Their exact representation should be determined by the machine class being studied.

IV. 4. Meta-Machines and Recursive Construction

A. 4.1. Definition by Construction Capability

A meta-machine is provisionally defined as a machine whose specified computational capabilities include constructing, composing, transforming, or orchestrating other machines.

Let M_0 be a constructor. Its output may be a machine M_1:

$$M_0 \xrightarrow{\mathcal{C}} M_1.$$

If M_1 is itself capable of constructing another machine, the process may continue:

$$M_0 \to M_1 \to M_2 \to \cdots.$$

This notation describes a possible sequence of construction events. It does not imply that every construction process is infinite, that every generated machine is a meta-machine, or that recursive construction must be unbounded.

A meta-machine may produce an individual component, a complete graph, or another construction system.

The distinction between machine construction and ordinary execution is important. A machine that generates a report during execution is not necessarily constructing a new machine. Construction requires the output to have a defined status as a machine description, architecture, or constructor under the adopted formal model.

B. 4.2. Levels of Construction

For analytical purposes, three levels are distinguished.

Level 1: Computational execution. A machine executes a computation or explores possible computational paths.

Level 2: Machine construction. A constructor generates or assembles machine descriptions from specifications and permitted components.

Level 3: Meta-construction. A system constructs or transforms the mechanisms through which further machines are constructed.

These levels are conceptual distinctions, not a claim that all systems must be organized into a strict hierarchy. A single implementation may combine multiple levels.

C. 4.3. Dynamic Architecture Transformation

A machine architecture may change in response to an event, resource constraint, or construction decision.

Let G_t denote the architecture at time t, and let e_t denote an event or condition. A transformation operator can be represented as:

$$G_{t+1} = T(G_t, e_t).$$

A transformation may add or remove a component, modify a dependency, change an orchestration rule, or replace a subgraph.

The key research problem is not merely how to modify a graph, but how to constrain the set of permitted transformations.

Let $I(G)$ be a required invariant. A desired preservation property is:

$$I(G) \wedge \mathrm{Pre}_T(G, e) \Rightarrow I(T(G, e)),$$

where Pre_T represents the preconditions for the transformation.

If this implication is formally established for every permitted transformation, it provides a basis for proving invariant preservation across a sequence of transformations, subject to the stated assumptions.

This is particularly important for dynamically constructed systems that must preserve interface compatibility, access-control rules, or other safety properties.

V. 5. Deductive Construction and Probabilistic Synthesis

A. 5.1. Deductive Construction

Deductive construction uses explicit rules to derive a machine or architecture from a specification.

Let D be a specification and $\mathcal{R}$ a set of construction rules. We write:

$$D, \mathcal{R} \vdash M$$

to indicate that M is derivable under the chosen construction system.

This notation requires a concrete inference system before it can support a mathematical theorem. A complete formulation must specify the syntax of machine descriptions, the rules of inference, and the meaning of derivability.

The intended objective is to make the construction process auditable and amenable to proof. A derivation should explain how the candidate was obtained and which rules were used.

A successful derivation does not automatically prove every behavioral property of the resulting implementation. It establishes only the conclusions supported by the construction rules and their semantics.

B. 5.2. Probabilistic Synthesis

A second mode generates candidates according to a probability distribution:

$$M \sim P(M \mid D, \Theta),$$

where D is the specification and Θ represents the parameters or information used by the synthesis mechanism.

This model accommodates exploratory construction in which the system generates several plausible architectures and evaluates them against requirements.

The probability of generating a candidate must be distinguished from the candidate's validity. A highly probable candidate may violate a requirement, while a valid candidate may have low probability under the synthesis model.

Accordingly, a probabilistic construction process may follow the conceptual cycle:

$$\text{Generate} \rightarrow \text{Verify} \rightarrow \text{Select} \rightarrow \text{Compose}.$$

The cycle may be repeated when no acceptable candidate is found.

This process does not guarantee that a valid candidate exists, that the generator will discover one, or that the search will terminate. Such guarantees require additional assumptions and methods.

C. 5.3. Combining Deduction and Probabilistic Exploration

The two construction modes need not be mutually exclusive.

A probabilistic mechanism may propose candidate architectures, while deductive rules or external verifiers reject candidates that violate mandatory constraints.

The combined system can distinguish:

- candidate generation;
- candidate ranking;
- structural validation;
- behavioral verification;
- selection for execution.

This separation makes it possible to explore alternative architectures without treating probabilistic confidence as a substitute for formal correctness.

The research challenge is to determine how the construction system represents uncertainty, how candidate evaluation is performed, and how the final selection relates to formal requirements.

VI. 6. External Formal Verification

A. 6.1. Separation of Construction and Verification

The proposed framework separates the construction subsystem from the verification subsystem.

Let:

$$M = \mathcal{C}(D)$$

represent a candidate produced from specification D. Let Φ denote the set of properties required of that candidate. A verifier evaluates:

$$v = \mathcal{V}(M, \Phi).$$

The verifier's result should distinguish at least three outcomes:

$$v \in \{\text{verified}, \text{rejected}, \text{inconclusive}\}.$$

The meaning of *verified* must be tied to a particular method, model, property, and set of assumptions. It should not be interpreted as proof that the machine is universally correct or free of all possible errors.

Likewise, a rejected candidate may violate a requirement, while an inconclusive result may indicate that the method cannot establish the property within its supported scope.

This separation allows different verification methods to be applied without embedding all verification logic in the construction mechanism.

B. 6.2. Hoare-Style Reasoning for Composed Machines

Hoare logic provides a foundation for reasoning about program preconditions and postconditions [4]. A construction framework may use contracts to describe the requirements and guarantees of individual components.

For a component A:

$$\{P_A\}\ A\ \{Q_A\}.$$

For a downstream component B, composition may require:

$$Q_A \Rightarrow P_B.$$

This condition expresses that the guarantee produced by A is sufficient to establish the precondition required by B.

For a graph containing parallel branches, however, the local contracts alone may be insufficient. The framework must account for shared state, interference, synchronization, and dependencies between branches.

A suitable verification architecture may therefore combine Hoare-style contracts with methods for concurrent programs or graph-based transition systems.

The paper does not propose a completed extension of Hoare logic. Instead, it identifies the extension of contract reasoning to nonlinear and dynamically changing machine graphs as a research problem.

C. 6.3. Graphs, Hypergraphs, and Behavioral Semantics

A graph can represent pairwise relations between components. A hypergraph may represent a relation involving multiple source or target components.

For example, a synchronization operation may require outputs from both B and C before D can execute.

$$\{B, C\} \to D.$$

A hyperedge can express this dependency directly. Nevertheless, the representation itself does not define execution semantics or prove correctness.

The framework must specify whether a relation represents data dependence, synchronization, control flow, or another interaction. It must also define how the corresponding relation is interpreted by the runtime and verifier.

Graph rewriting may be appropriate for specifying structural transformations, while model checking may be appropriate for certain behavioral properties. Type systems and contract checking may address interface compatibility.

The verification method should be selected according to the property to be established, rather than assuming that one mathematical structure solves all verification problems.

D. 6.4. Verification Profiles

The framework may assign a verification profile to each machine or architecture.

A profile specifies:

1) the required properties;
2) the applicable verification methods;
3) the assumptions under which each method operates;
4) the expected evidence or result;
5) the conditions under which the candidate may proceed to execution.

For example, a profile may require structural consistency, interface compatibility, selected safety properties, and runtime monitoring.

The profile should not collapse testing, static analysis, formal proof, and runtime observation into a single undifferentiated notion of correctness. Each provides different kinds of evidence and guarantees.

E. 6.5. Probabilistic Properties

Some systems require quantitative statements about risk or failure probability.

A requirement may take the form:

$$P(\text{failure}) \leq \varepsilon.$$

Such a statement is meaningful only relative to a defined probabilistic model, assumptions about events, and an interpretation of failure.

Probabilistic verification is therefore complementary to, rather than interchangeable with, deterministic proof. The construction framework must preserve this distinction when it evaluates probabilistically generated candidates.

VII. 7. Proposed Architecture for the Construction Agent

A. 7.1. Almudaya Machine Construction Agent

The proposed Proof-of-Concept system is tentatively named **Almudaya Machine Construction Agent (AMCA)**.

AMCA is intended to demonstrate an operational construction workflow in which an agent receives a specification, builds a machine or graph description through an explicit scaffold, submits the candidate for external verification, and revises rejected candidates when possible.

The prototype is not intended to establish the entire mathematical framework. Its purpose is to test whether the proposed abstractions can guide a concrete construction process.

B. 7.2. System Architecture

The proposed architecture contains the following subsystems:

1) **Specification Interpreter:** extracts requirements, constraints, and acceptance criteria.
2) **Construction Agent:** proposes a machine or architecture based on the specification.
3) **Construction Scaffold:** defines permitted components, composition rules, interface contracts, and structural constraints.
4) **Architecture Builder:** converts the proposed description into a graph or executable configuration.
5) **External Verification System:** checks structural properties and selected contracts.
6) **Revision Mechanism:** revises rejected candidates based on verification results.
7) **Runtime and Orchestrator:** executes architectures that meet the required acceptance criteria.
8) **Evidence and Logging System:** records construction steps, verification results, and execution outcomes.

A simplified workflow is:

$$D \to C \to G \to V$$

where D is the specification, C is the construction process, G is the generated architecture, and V is verification.

If the candidate is accepted under the required verification profile, it may proceed to execution. If it is rejected, the system may revise the candidate or report that the specification cannot be satisfied within the available construction rules.

The system should not assume that every rejection can be repaired automatically.

C. 7.3. Construction Scaffold

The scaffold is a central element of the prototype. It provides an explicit structure within which the agent must operate.

A preliminary directory layout is:

```
amca/
+-- README.md
+-- pyproject.toml
|
```

```
+-- specification/
|   +-- problem.md
|   +-- requirements.yaml
|   `-- acceptance_criteria.yaml
|
+-- construction/
|   +-- machine_model.md
|   +-- construction_rules.yaml
|   +-- composition_rules.yaml
|   +-- transformation_rules.yaml
|   `-- scaffold_schema.yaml
|
+-- architecture/
|   +-- architecture.yaml
|   +-- graph.yaml
|   `-- contracts/
|
+-- verification/
|   +-- verification_plan.yaml
|   +-- structural_verifier.py
|   +-- contract_verifier.py
|   `-- verification_report.schema.yaml
|
+-- runtime/
|   +-- engine.py
|   +-- orchestrator.py
|   `-- state_manager.py
|
+-- agents/
|   +-- construction_agent.py
|   +-- verification_agent.py
|   `-- revision_agent.py
|
+-- experiments/
|   +-- experiment_001.yaml
|   `-- results/
|
`-- tests/
    +-- test_structure.py
    +-- test_contracts.py
    +-- test_composition.py
    `-- test_meta_construction.py
```

This directory structure is illustrative and may be refined during implementation.

YAML is proposed as a declarative representation because it can express structured configuration and machine-readable relationships. Markdown can document the construction rules, design rationale, and requirements.

However, YAML is not the mathematical foundation of the framework. A parser and validator must translate its descriptions into an internal representation with defined semantics. A more specialized language may be introduced later if the requirements exceed the expressive or validation capabilities of the initial representation.

The scaffold must constrain more than directory names and file locations. It should define admissible components, interface schemas, construction rules, verification requirements, and conditions for accepting an architecture.

D. 7.4. Initial Construction Experiment

The first experiment will ask AMCA to construct a system that:

1) receives a data input;
2) validates the input;
3) sends the validated data to two analysis components;
4) waits for the required outputs;
5) combines the outputs;
6) produces a report.

The expected graph is:

$$G_1 : I \to V \to \begin{cases} A_1 \\ A_2 \end{cases} \to F \to O.$$

The verifier should examine whether all required nodes and dependencies are present, whether the component interfaces are compatible, and whether the aggregation component receives the outputs required by its contract.

A negative test should deliberately omit one required dependency. The verifier should reject the candidate if the omission violates the specified constraints.

This experiment tests nonlinear construction and external validation. It does not, by itself, establish the correctness of a general machine-construction theory.

E. 7.5. Meta-Machine Construction Experiment

The second experiment will ask AMCA to construct a system that accepts task specifications, selects components, assembles a graph, verifies the graph, and prepares it for execution.

The resulting architecture is itself a machine constructor.

The experiment will investigate whether the construction process can operate at more than one level of abstraction:

$$M_0 \to M_1 \to M_2,$$

where M_0 constructs M_1, and M_1 can construct M_2.

The goal is not unrestricted recursion. The prototype should impose explicit limits on construction depth, resource use, and the classes of machines that may be generated.

A successful experiment would demonstrate recursive construction capability within the defined prototype scope, not universal self-construction or unrestricted machine generation.

VIII. 8. Experimental Plan and Evaluation

The PoC should be evaluated through controlled experiments rather than demonstrations alone.

A. 8.1. Experiment Categories

E1 — Linear Construction. Generate a machine consisting of sequential components from a specification.

E2 — Nonlinear Construction. Generate a graph containing branching, parallel operations, and aggregation.

E3 — Meta-Machine Construction. Generate a machine whose specified function includes constructing another machine or graph.

E4 — Probabilistic Synthesis. Generate multiple candidates, evaluate them, and select a candidate satisfying the required constraints.

E5 — Dynamic Transformation. Modify an existing architecture in response to a defined event and examine whether required invariants are preserved.

These experiments are proposed future work. No empirical outcomes are claimed in this paper.

B. 8.2. Evaluation Metrics

The following metrics are proposed:

- **Structural validity rate:** proportion of generated candidates satisfying the defined structural constraints.
- **Contract satisfaction rate:** proportion of candidates whose required interfaces and contracts pass verification.
- **Construction success rate:** proportion of tasks for which a candidate satisfying the acceptance criteria is produced.
- **Revision count:** number of construction or repair iterations before acceptance or termination.
- **Construction cost:** computational resources and elapsed time required to generate and verify candidates.
- **Meta-construction success:** proportion of meta-construction tasks completed within the specified limits.
- **Invariant preservation:** whether permitted transformations preserve the selected invariants under the stated assumptions.
- **Probabilistic candidate diversity:** variation among generated candidates, measured using a representation-appropriate metric.

Metrics must be defined operationally before experiments begin. For example, candidate diversity cannot be measured consistently until the representation and equivalence criteria for architectures have been specified.

C. 8.3. Baselines and Ablation Studies

To assess whether the scaffold and verification layers contribute to performance, future experiments should compare:

1) construction with and without explicit construction rules;
2) construction with and without structural verification;
3) construction with and without contract verification;
4) single-candidate generation against multi-candidate probabilistic synthesis;
5) fixed architecture against controlled dynamic transformation.

Where feasible, a baseline using ordinary agent-generated code or workflow configuration should be included.

The comparison should evaluate not only task completion but also structural validity, repair effort, reproducibility, and the quality of verification evidence.

IX. 9. Formalization Agenda

The operational prototype and the mathematical framework should develop together. Several formal questions remain unresolved.

A. 9.1. Definition of a Machine Object

The theory must define the admissible class of machine descriptions and the semantics associated with each description.

It must also determine whether two different descriptions may represent equivalent machines and how such equivalence affects composition and verification.

B. 9.2. Algebra of Construction

The construction operators require explicit definitions.

Important questions include:

- Under what conditions is composition defined?
- Is composition associative for the chosen class of machines?
- Does an identity construction exist?
- Which operations preserve type or interface compatibility?
- Can transformations be composed?
- What conditions guarantee closure within the admitted machine class?

These properties must be established rather than assumed.

The algebraic structure need not be a group. Some construction operations may be irreversible, partial, or non-invertible. A suitable formalism must accommodate the operations required by the machine class instead of forcing them into an inappropriate algebraic structure.

C. 9.3. Compositional Correctness

A central objective is to establish conditions under which valid components can be composed into a valid architecture.

A desired result may take the general form:

$$\left(\bigwedge_{i=1}^{n} \mathrm{Contract}(M_i)\right) \wedge \mathrm{Compatible}(G) \Rightarrow \mathrm{Property}(G).$$

This expression is a research objective, not a theorem established by the present paper. Its validity depends on the definition of contracts, compatibility, graph semantics, and the target property.

D. 9.4. Dynamic Invariant Preservation

The framework must determine how the verifier evaluates transformations that modify a running architecture.

Possible approaches include proving that each permitted transformation preserves an invariant, verifying each candidate architecture before activation, or combining static verification with runtime monitoring.

The appropriate approach may differ depending on whether the architecture is immutable during execution, reconfigured at controlled boundaries, or modified continuously.

E. 9.5. Probabilistic Construction Semantics

The probabilistic construction operator requires a formal interpretation of candidate distributions, selection criteria, and failure conditions.

Further work must determine whether the synthesis mechanism is probabilistic in its transition semantics, stochastic in its search strategy, or merely uses a probabilistic model to rank candidates. These alternatives have different mathematical implications.

F. 9.6. Relationship to NUTM

NUTM remains a conceptual source of inspiration for branching possibilities. A more precise relationship would require defining how computation branches correspond, if at all, to construction alternatives.

The framework should not assume that nondeterministic computation directly entails machine synthesis, nor that universal simulation is equivalent to recursive machine construction.

A formal treatment should identify which properties are inherited from the underlying computational model and which require independent construction operators and verification rules.

X. 10. Discussion

The proposed framework is intended to shift the focus from systems that merely execute predefined computational structures to systems that explicitly construct and transform those structures.

This perspective has potential relevance to agentic software engineering, automated workflow generation, adaptive software architectures, program synthesis, and systems that orchestrate heterogeneous computational services.

Nevertheless, several distinctions must remain clear.

First, a machine description is not identical to its executable implementation. A description may be structurally valid while its implementation fails to realize the specified semantics.

Second, successful construction is not equivalent to formal correctness. Correctness depends on the property being examined, the formal model, and the verification method.

Third, probabilistic synthesis is not itself a proof mechanism. Candidate generation and candidate verification must remain distinct.

Fourth, a graph representation does not fully define concurrency, communication, synchronization, or runtime behavior. These require explicit semantics.

Finally, a scaffold is not a mathematical theory. It operationalizes construction rules and constraints, while the formal model explains what those rules mean and which properties they preserve.

The separation between representation, construction, verification, and execution is therefore a central design principle of the proposed research program.

The framework also raises a broader question concerning the boundary between a constructor and the object it constructs. A construction system may initially be a fixed program, but its output may itself contain construction capabilities. This recursive relationship deserves formal treatment because it connects the structure of a machine with the process through which machines are produced.

Whether these ideas require a new mathematical formalism or can be expressed through a carefully selected combination of existing theories remains an open question. Establishing that distinction is part of the research agenda rather than a conclusion of the present paper.

XI. 11. Limitations

This paper has several limitations.

First, the machine model remains abstract. A complete formal definition of machine descriptions, construction operators, and execution semantics has not yet been established.

Second, the proposed mathematical expressions are preliminary. They identify candidate abstractions but do not constitute a complete algebra or a proven calculus of machine construction.

Third, the verification architecture is conceptual. No general verification procedure is claimed to establish all required properties for arbitrary nonlinear or dynamically changing machine graphs.

Fourth, the AMCA prototype has been proposed but not implemented or experimentally evaluated in this paper. Consequently, no claims concerning performance, scalability, construction accuracy, or verification success are supported by empirical results.

Fifth, the relationship between the proposed framework and existing research must be developed through a more comprehensive literature review. In particular, program synthesis, graph transformation, compositional verification, reflective computation, workflow synthesis, and self-adaptive systems may already address important parts of the problem.

These limitations are intentional boundaries for the present stage of the research. The purpose is to establish a coherent conceptual framework and identify the formal and experimental work required to evaluate it.

XII. 12. Conclusion and Future Work

This paper has introduced an initial framework for treating computational machines as constructible objects that can be composed into graphs, transformed into new architectures, and used in recursive machine-construction processes.

Nondeterministic Universal Turing Machines provide an inspiration for considering branching possibilities, but the proposed framework distinguishes computational branching from the construction of machine descriptions. This distinction motivates the introduction of explicit construction operators and a separate verification architecture.

The framework identifies two complementary construction modes: deductive construction, which uses explicit rules, and probabilistic synthesis, which explores candidate architectures. It further proposes that construction and verification should be separate but interoperable subsystems connected through specifications, contracts, and verification evidence.

A preliminary Proof-of-Concept system, AMCA, has been outlined. It uses a construction scaffold to generate machine descriptions and graphs, an external verification layer to check selected properties, and a runtime to execute accepted candidates. YAML is proposed as an initial declarative representation rather than as the mathematical foundation of the framework.

The immediate next step is to define a minimal formal machine model, specify a small set of construction and composition operators, and implement a prototype capable

of generating and verifying linear and nonlinear architectures. Subsequent work should investigate recursive meta-construction, dynamic invariant preservation, and probabilistic candidate synthesis.

The broader research objective is to determine whether a formal construction framework can support the systematic generation of machines and meta-machines while retaining explicit, testable validity conditions.

At this stage, the framework is a research proposal rather than a completed theory. Its value will depend on the precision of its formal definitions, the strength of its verification results, and the evidence produced by its implementation and evaluation.

REFERENCES

[1] National Institute of Standards and Technology, "Universal Turing Machine," *Dictionary of Algorithms and Data Structures*. Available at: https://xlinux.nist.gov/dads/HTML/universalTur.html.

[2] National Institute of Standards and Technology, "Nondeterministic Turing Machine," *Dictionary of Algorithms and Data Structures*. Available at: https://xlinux.nist.gov/dads/HTML/nondetermTuringMach.html.

[3] T. Kurabayashi, H. Tanno, H. Kirinuki, and T. Oshima, "A Survey of Program Synthesis," *Journal of Software Science and Technology*, vol. 40, no. 2, pp. 95–128, 2023. doi: 10.11309/jssst.40.2_95.

[4] C. A. R. Hoare, "An Axiomatic Basis for Computer Programming," *Communications of the ACM*, vol. 12, no. 10, pp. 576–580, 1969. doi: 10.1145/363235.363259.